

05

Pisanje sopstvenih funkcija

Kod koji se ponavlja pakujemo u imenovanu celinu i od malih blokova gradimo veće programe

Isti kod na više mesta? Napravite funkciju

```
my_dna = "ACTGATCGATTACGTATAGTATTTGCTATCATACATATATCGATGCGTTTCAT"  
length = len(my_dna)  
a_count = my_dna.count('A')  
t_count = my_dna.count('T')  
at_content = (a_count + t_count) / length  
print("AT content is " + str(at_content))
```

Četiri istaknuta reda

računaju AT sadržaj. Gde god nam on treba, morali bismo da ih prepíšemo bez greške. Zgodnija bi bila funkcija `get_at_content`, kao `print`, `len` ili `open`.

Sopstvena funkcija

- kod se piše jednom, koristi više puta
- izmena se pravi na jednom mestu
- delovi programa se razvijaju nezavisno
- ista funkcija radi i u drugim programima

Definisanje funkcije

```
def get_at_content(dna):  
    length = len(dna)  
    a_count = dna.count('A')  
    t_count = dna.count('T')  
    at_content = (a_count + t_count) / length  
    return at_content
```

Pre pisanja odredite ulaz i izlaz: jedan argument tipa string (DNK), a povratna vrednost je broj.

def

„definiši”: počinje definiciju

get_at_content

ime funkcije, proizvoljno

(dna)

argument: promenljiva koja dobija vrednost pri pozivu

: i blok

telo funkcije, uvučeno kao kod petlje

return

vrednost koju funkcija vraća

Definicija nije poziv

```
# poziv bez čuvanja rezultata: vrednost se gubi
get_at_content("ATGACTGGACCA")
# sačuvajte rezultat u promenljivoj...
at_content = get_at_content("ATGACTGGACCA")
# ...ili ga odmah iskoristite
print("AT content is " + str(get_at_content("ATGACTGGACCA")))
```

Argument čeka vrednost

`dna` nema vrednost pri definiciji. Dobija je pri svakom pozivu, i to svaki put drugu, kao promenljiva petlje.

Opseg (scope)

Promenljive napravljene u funkciji ne postoje van nje:

```
print(a_count) → NameError: name 'a_count' is not
defined
```

Prva upotreba otkriva dva problema

```
# get_at_content: prva verzija funkcije
my_at_content = get_at_content("ATGCGCGATCGATCGAATCG")
print(str(my_at_content))
print(get_at_content("ATGCATGCAACTGTAGC"))
print(get_at_content("aactgtagctagctagcagcgta"))
```

IZLAZ

```
0.45
0.5294117647058824
0.0
```

Previše decimala

Drugi poziv daje 0.5294117647058824. Dovoljne su dve decimale.

Mala slova daju 0.0

`count('A')` ne broji malo a, pa je rezultat za treću sekvencu pogrešan.

Poboljšanje funkcije

```
def get_at_content(dna, sig_figs):  
    length = len(dna)  
    a_count = dna.upper().count('A')  
    t_count = dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    return round(at_content, sig_figs)
```

```
test_dna = "ATGCATGCAACTGTAGC"  
print(get_at_content(test_dna, 1))  
print(get_at_content(test_dna, 2))  
print(get_at_content(test_dna, 3))
```

Šta je popravljeno

- Previše decimala (0.5294117647058824): **round** zaokružuje rezultat
- Mala slova davala su 0.0: **upper()** pre brojanja
- Broj decimala postaje drugi argument, **sig_figs**

0.5
0.53
0.529

IZLAZ

Promenili smo definiciju funkcije, a da nismo dirali kod koji je koristi.

To je enkapsulacija: program delimo na male delove na kojima radimo nezavisno, a svaki je dovoljno mali da ga ceo razumemo.

Dve navike koje kvare funkcije

Oslanjanje na spoljnu promenljivu

```
def get_at_content():  
    length = len(dna)  
    # ... ostatak kao ranije  
dna = "ACTGATCGATCG"  
print(get_at_content())
```

Radi samo ako negde postoji promenljiva **dna**, pa delovi koda više nisu nezavisni. Spoljne promenljive pretvorite u argumente.

Ispis umesto povratne vrednosti

```
def print_at_content(dna):  
    # ... računanje kao ranije  
    print(str(round(at_content, 2)))
```

„Izračunaj i ispiši” su dva posla. Neka funkcija radi jedan posao i vrati rezultat; ispis, upis u fajl ili dalju računicu radi kod koji je poziva.

Imenovani argumenti i podrazumevane vrednosti

Imenovani argumenti

```
get_at_content("ATCGTGACTCG", 2)
get_at_content(dna="ATCGTGACTCG", sig_figs=2)
get_at_content(sig_figs=2, dna="ATCGTGACTCG")
get_at_content("ATCGTGACTCG", sig_figs=2)
# greška: posle imenovanog ne ide običan
get_at_content(dna="ATCGTGACTCG", 2)
```

Prva četiri poziva su ista. Redosled imenovanih argumenata nije bitan, a poziv je čitljiviji.

Podrazumevane vrednosti

```
def get_at_content(dna, sig_figs=2):
    # ... telo kao ranije
get_at_content("ATCGTGACTCG")
get_at_content("ATCGTGACTCG", 3)
get_at_content("ATCGTGACTCG", sig_figs=4)
```

IZLAZ

```
0.45
0.455
0.4545
```

Podrazumevana vrednost ima smisla samo kad postoji razuman izbor; za ^{dna} ne postoji.

Testiranje funkcija: assert

Ako uslov posle `assert` nije tačan, program staje sa `AssertionError`. Kao pozitivna kontrola u laboratoriji.

```
assert get_at_content("A") == 1
assert get_at_content("G") == 0
assert get_at_content("ATGC") == 0.5
assert get_at_content("AGG") == 0.33
assert get_at_content("AGG", 1) == 0.3
assert get_at_content("AGG", 5) == 0.33333
# nepoznate baze N ne bi smele da utiču
assert get_at_content("ATGCNNNNNNNNN") == 0.5
```

```
def get_at_content(dna, sig_figs=2):
    # ispravka: izbaciti N pre računanja
    dna = dna.replace('N', '')
    # ... ostatak kao ranije
```

- Pokazuju da li je greška u funkciji ili u kodu koji je poziva
- Posle izmene funkcije ponovo pokrenite testove
- Dokumentuju očekivani izlaz za dati ulaz
- Proveravaju neobične ulaze, npr. baze N

Od konkretnog koda do funkcije

Konkretan primer

```
protein = "MSRSLLLRFLFLLLLPPLP"  
aa = "R"  
aa_count = protein.count(aa)  
protein_length = len(protein)  
percentage = aa_count * 100 / protein_length  
print(percentage)  # 10.0
```

Kao funkcija

```
def get_aa_percentage(protein, aa):  
    aa_count = protein.count(aa)  
    protein_length = len(protein)  
    percentage = aa_count * 100 / protein_length  
    return percentage
```

```
assert get_aa_percentage("MSRSLLLRFLFLLLLPPLP", "M") == 5  
assert get_aa_percentage("MSRSLLLRFLFLLLLPPLP", "r") == 10  
assert get_aa_percentage("msrslllrflfllllpplp", "L") == 50  
assert get_aa_percentage("MSRSLLLRFLFLLLLPPLP", "Y") == 0
```

Test pada: velika i mala slova

```
assert get_aa_percentage("MSRLLLLRFLLFLLLLPPLP", "r") == 10
```

`AssertionError`

```
def get_aa_percentage(protein, aa):  
    # oba ulaza u velika slova  
    protein = protein.upper()  
    aa = aa.upper()  
    aa_count = protein.count(aa)  
    protein_length = len(protein)  
    percentage = aa_count * 100 / protein_length  
    return percentage
```

Poruka kaže koji test je pao: protein je velikim, a oznaka malim slovima.

Sledeći test, sa proteinom malim slovima, pao bi iz istog razloga.

Rezultat `upper()` upisujemo nazad u istu promenljivu. Sada prolaze svi testovi.

Lista aminokiselina: zbir u petlji

```
protein = "MSRSLLLRFLFLLLLPPLP"
aa_list = ['M', 'L', 'F']
total = 0
for aa in aa_list:
    print("counting number of " + aa)
    aa = aa.upper()
    aa_count = protein.count(aa)
    total = total + aa_count
    print("running total is " + str(total))
percentage = total * 100 / len(protein)
print("final percentage is " + str(percentage))
```

IZLAZ

```
counting number of M
running total is 1
counting number of L
running total is 11
counting number of F
running total is 13
final percentage is 65.0
```

total pravimo pre petlje i povećavamo ga u svakom krugu: zbir koji raste.

Funkcija sa podrazumevanom listom

`TypeError: get_aa_percentage() missing 1 required positional argument: 'aa_list'`

```
def get_aa_percentage(protein, aa_list=['A', 'I', 'L', 'M', 'F', 'W', 'Y', 'V']):  
    protein = protein.upper()  
    protein_length = len(protein)  
    total = 0  
    for aa in aa_list:  
        aa = aa.upper()  
        aa_count = protein.count(aa)  
        total = total + aa_count  
    percentage = total * 100 / protein_length  
    return percentage
```

Bez drugog argumenta funkcija nije mogla da se pozove.

Podrazumevana vrednost: lista hidrofobnih aminokiselina. Sada svi testovi prolaze.

Rezime i vežbe

Šta smo naučili

- def, argumenti, telo i return
- Definicija nije poziv
- Opseg: promenljive funkcije žive samo u njoj
- Enkapsulacija i „jedan posao”
- Imenovani i podrazumevani argumenti
- Testiranje sa assert

Vežbe: procenat aminokiselina

1. Funkcija prima protein i oznaku aminokiseline i vraća njen procenat u proteinu:

```
assert my_function("MSRSLLLRFLFLLLLPPLP", "M") == 5
assert my_function("MSRSLLLRFLFLLLLPPLP", "L") == 50
```

2. Umesto jedne prima listu aminokiselina; bez liste vraća procenat hidrofobnih (A, I, L, M, F, W, Y, V):

```
assert my_function("MSRSLLLRFLFLLLLPPLP", ['M', 'L']) == 55
assert my_function("MSRSLLLRFLFLLLLPPLP") == 65
```